TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN

CÔNG NGHỆ PHẦN MỀM

Báo cáo Assignment 2

BrewLite

Ứng dụng đặt đồ uống và thanh toán không tiền mặt

Khai thác yêu cầu, mô hình hóa, kiến trúc và triển khai

GVHD: Từ Lãng Phiêu

SV:	Trương Trọng Phúc	MSSV: 3124410287
	Chế Văn Nam	MSSV: 3124410217
	Đỗ Đạt	MSSV: 3124410054
	Lại Đình Nguyên	MSSV: 3124410235
	Lê Trung Hưng	MSSV: 3124410128
	Đỗ Hữu Đăng Khoa	MSSV: 3124410155
	Phạm Khánh Nguyên	MSSV: 3124410239

TP. HỒ CHÍ MINH, THÁNG 11/2026

Mục lục

1 Task 1 – Khai thác và đặc tả yêu cầu **3**

1.1 Task 1.1 – Bối cảnh, stakeholder và phạm vi . . . 3

1.1.1 Vấn đề nghiệp vụ . . . 3

1.1.2 Stakeholder và kỳ vọng . . . 3

1.1.3 Mục tiêu và phạm vi . . . 3

1.1.4 Giả định, ràng buộc và rủi ro ban đầu . . . 4

1.2 Task 1.2 – Yêu cầu chức năng và phi chức năng . . . 4

1.2.1 Yêu cầu chức năng . . . 5

1.2.2 Yêu cầu phi chức năng . . . 9

1.2.3 Sơ đồ use-case tổng quát . . . 12

1.2.4 Các giả định cần nhóm xác nhận . . . 12

1.3 Task 1.3 – Đặc tả use-case trọng tâm . . . 12

1.3.1 UC-01 – Đặt và thanh toán đơn hàng . . . 12

1.3.2 Sơ đồ use-case của UC-01 . . . 13

1.3.3 Luồng chính – thanh toán thành công . . . 13

1.3.4 Luồng thay thế . . . 14

1.3.5 Luồng ngoại lệ . . . 14

1.3.6 Liên kết yêu cầu và tiêu chí chấp nhận . . . 15

2 Task 2 – Mô hình hóa hệ thống **16**

2.1 Task 2.1 – Activity diagram . . . 16

2.1.1 Luồng đặt và thanh toán . . . 16

2.1.2 Kiểm tra giỏ và giữ nguyên liệu . . . 17

2.1.3 Thanh toán thẻ mô phỏng và chống xử lý lặp . . . 18

2.1.4 VietQR, webhook và đối soát . . . 19

2.2 Task 2.2 – Sequence diagram . . . 20

2.2.1 Luồng UC-01 từ giỏ đến biên nhận . . . 20

2.2.2 Tạo đơn và tạm giữ nguyên liệu . . . 21

2.2.3 Chống xử lý lặp khi thanh toán thẻ . . . 22

2.2.4 Thanh toán VietQR và đối soát PayOS . . . 23

2.3 Task 2.3 – Class diagram . . . 24

2.3.1 Mục tiêu và phạm vi mô hình hóa mức Domain . . . 24

2.3.2 Sơ đồ lớp miền (Domain Class Diagram) . . . 24

2.3.3 Mô tả các thực thể chính và thuộc tính . . . 25

2.3.4 Quan hệ và bội số (Multiplicity) . . . 25

3 Task 3 – Thiết kế kiến trúc **25**
3.1 Task 3.1 – Phương pháp và quyết định kiến trúc 25
3.2 Task 3.2 – Deployment diagram 26

4 Task 4 – Triển khai Sprint 1 **26**
4.1 Task 4.1 – Thiết lập repository 26
4.2 Task 4.2 – Quản lý tài liệu 26
4.3 Task 4.3 – MVP Sprint 1 27
4.4 Task 4.4 – Continuous Integration 27

5 Task 5 – Triển khai Sprint 2 và bàn giao **28**
5.1 Task 5.1 – Hoàn thiện MVP 28
5.2 Task 5.2 – Tích hợp và kiểm thử 28
5.3 Task 5.3 – Continuous Delivery/Deployment 29
5.4 Task 5.4 – Triển khai và bàn giao 29

1 Task 1 – Khai thác và đặc tả yêu cầu

1.1 Task 1.1 – Bối cảnh, stakeholder và phạm vi

1.1.1 Vấn đề nghiệp vụ

Khách mua đồ uống cần biết món nào còn bán, giá theo size và topping, và liệu cửa hàng có đủ nguyên liệu trước khi đặt. Nếu giá hoặc tồn kho chỉ được tính trên trình duyệt, đơn có thể sai tiền hoặc bán vượt lượng nguyên liệu sẵn có. Cửa hàng cần nhận đơn đã thanh toán, theo dõi pha chế và đối chiếu trạng thái đơn; khách cần nhận mã đơn và theo dõi tiến độ để lấy đồ tại quầy. BrewLite giải quyết luồng từ xem menu, chọn món, giỏ hàng, tạo đơn, thanh toán đến theo dõi và hoàn tất đơn trên một hệ thống web.

1.1.2 Stakeholder và kỳ vọng

Đối tượng	Mục tiêu và kỳ vọng	Cách xác nhận
Khách hàng	Xem món, tùy chọn, giá và khả năng đáp ứng; đặt, thanh toán, nhận mã và theo dõi đơn; dùng voucher/điểm đúng quy tắc.	Demo luồng mua và các ca giỏ rỗng, hết nguyên liệu, thanh toán lỗi.
Nhân viên quầy	Chỉ nhận đơn đã thanh toán; cập nhật pha chế, sẵn sàng và hoàn thành đơn kịp thời.	Demo danh sách đơn và các bước chuyển trạng thái hợp lệ.
Quản lý cửa hàng	Kiểm soát nguyên liệu, công thức, chương trình ưu đãi và doanh thu; tránh bán vượt tồn hoặc ghi nhận thanh toán lặp.	Đối soát tồn, trạng thái đơn, giao dịch và quyền quản trị.
Nhóm phát triển	Có yêu cầu rõ, phân ranh frontend/API/DB, quy trình tích hợp và kiểm thử lặp lại.	Build/test qua CI, tài liệu triển khai và ma trận truy vết.
Giảng viên / khách hàng nghiệm thu	Đánh giá MVP, tính nhất quán của yêu cầu, sơ đồ và kết quả kiểm thử.	Demo theo tiêu chí chấp nhận và đối chiếu ma trận truy vết.
Đơn vị thanh toán (PayOS)	Nhận đúng mã đơn, số tiền và URL webhook; cung cấp trạng thái giao dịch để xác minh.	Kiểm thử webhook, đối soát link và xử lý tín hiệu lặp.

1.1.3 Mục tiêu và phạm vi

Mục tiêu MVP: Khách hoàn tất luồng xem menu → chọn size/topping → giỏ → tạo đơn → thanh toán → theo dõi đơn; nhân viên xử lý đơn đã thanh toán. Giá, quyền và lượng nguyên liệu được máy chủ kiểm tra; đơn và thanh toán được ghi nhất quán.

Trong phạm vi MVP:

- Menu, chi tiết món, giá theo size/topping, giỏ và kiểm tra khả năng đáp ứng theo công thức nguyên liệu.

- Đăng ký/đăng nhập, tạo đơn có mã và thời hạn thanh toán; thanh toán thẻ mô phỏng và luồng VietQR qua PayOS khi đã cấu hình; lịch sử và trạng thái đơn.
- Quản trị đơn sau thanh toán; quản lý nguyên liệu, công thức và các khuyến mãi đang sử dụng.
- Next.js, NestJS, PostgreSQL, CI/CD và URL production qua HTTPS theo yêu cầu Assignment 2.

Phạm vi mở rộng: điểm thưởng, đổi voucher, báo cáo quản trị chi tiết, PWA/cải thiện trải nghiệm di động và kiểm thử tải ở quy mô lớn hơn dữ liệu demo.

Ngoài phạm vi: giao tận nơi, theo dõi tài xế, nhiều chi nhánh độc lập, tự đối soát mọi ngân hàng/ví và hoàn tiền ngân hàng tự động. Thanh toán thẻ dùng cổng mô phỏng. VietQR qua PayOS xác nhận giao dịch thật khi kênh thanh toán và webhook được cấu hình hợp lệ.

1.1.4 Giả định, ràng buộc và rủi ro ban đầu

Giả định: Khách nhận đồ tại quầy; sản phẩm có công thức theo size; nguyên liệu theo dõi tồn có lượng khả dụng bằng `onHand - reserved`; thời hạn giữ nguyên liệu của đơn chờ thanh toán là hai phút. Lượt tạo đơn thay thế sau hết hạn là quy tắc giao diện hiện tại và cần được nghiệm thu riêng.

Ràng buộc: Ba thành phần bắt buộc của đề là Next.js, NestJS và PostgreSQL. Web chạy trên Vercel, API trên Render, DB trên Supabase; secret chỉ đặt ở môi trường triển khai. API là nguồn tin cậy cho giá, quyền, tồn và trạng thái. Dữ liệu sản xuất không được dùng làm dữ liệu thử phá hủy.

Rủi ro:

- Tài nguyên free tier có thể ngủ hoặc giới hạn kết nối, khiến phản hồi đầu tiên chậm và webhook đến trễ.
- Webhook PayOS cần URL công khai và khóa xác minh của kênh thanh toán. Bản demo phân biệt thanh toán thẻ mô phỏng với giao dịch VietQR thật.
- Đặt đồng thời có thể tranh chấp nguyên liệu; tạo đơn và giữ tồn phải nằm trong transaction, có kiểm thử cạnh tranh.
- Sai lệch giữa cấu hình production, schema migration và mã nguồn có thể làm deploy lỗi; cần health check và phương án rollback.

1.2 Task 1.2 – Yêu cầu chức năng và phi chức năng

Tài liệu SRS rút gọn này chuyển từ bản thảo `task1_2.docx` của nhóm. Mỗi yêu cầu có mã định danh, mức ưu tiên MoSCoW và tiêu chí chấp nhận (AC) để truy vết sang thiết kế và kiểm thử. Đây là yêu cầu cần nghiệm thu, không phải tuyên bố tất cả đã được kiểm chứng. **Quy ước ưu tiên:** Must = bắt buộc; Should = nên có; Could = có thể có; Won't = ngoài phạm vi phiên bản này. **Quy tắc nghiệp vụ:** Máy chủ tính lại giá, quyền và nhu cầu nguyên liệu toàn giỏ. Bước kiểm tra giỏ chỉ đọc dữ liệu. Máy chủ giữ nguyên liệu trong transaction khi tạo đơn hợp lệ và tiêu thụ khi thanh toán thành công. Đơn `PENDING` có hạn hai phút. Khi hết hạn, máy chủ kiểm tra điều kiện hủy trước khi giải phóng nguyên liệu. Thanh toán thẻ dùng cổng mô phỏng. VietQR được ghi nhận sau khi API xác minh webhook hợp lệ hoặc đối soát trực tiếp với PayOS.

1.2.1 Yêu cầu chức năng

FR-01 – Xem danh sách menu **Ưu tiên:** Must.
Mô tả: Hệ thống cung cấp danh sách sản phẩm đang kinh doanh, có thông tin nhận diện và giá theo các size hợp lệ. Khách có thể lọc menu theo danh mục nếu chức năng lọc được bật.
AC-FR01.1: Với dữ liệu sản phẩm, API menu trả về từng sản phẩm đang kinh doanh cùng ID, tên, hình ảnh (nếu có), danh mục và danh sách size/giá hiện hành.

AC-FR01.2: Sản phẩm không kinh doanh không được trình bày như món có thể đặt; danh sách không có sản phẩm trả về trạng thái rỗng hợp lệ.

AC-FR01.3: Trang menu hiển thị trạng thái tải trong khi chờ API, trạng thái rỗng khi không có món và thông báo có thể nhận biết khi API lỗi; không thay lỗi bằng dữ liệu giả.

AC-FR01.4: Chọn một sản phẩm trên menu mở đúng trang chi tiết theo ID sản phẩm.

FR-02 – Xem chi tiết và tùy chỉnh sản phẩm **Ưu tiên:** Must.
Mô tả: Khách xem thông tin sản phẩm, chọn một size có giá và các topping hợp lệ trước khi thêm vào giỏ.
AC-FR02.1: Trang chi tiết hiển thị thông tin của đúng sản phẩm, giá theo các size mà sản phẩm hỗ trợ và các topping hiện còn khả dụng.

AC-FR02.2: Chọn size hoặc thay đổi số phần topping làm giá hiển thị được tính lại theo bảng giá hiện hành.

AC-FR02.3: Không cho chọn size không có giá, topping không khả dụng hoặc topping cho sản phẩm không hỗ trợ topping.

AC-FR02.4: ID sản phẩm không tồn tại hiển thị trạng thái không tìm thấy; lỗi tải dữ liệu được thông báo thay vì hiển thị một sản phẩm khác.

FR-03 – Quản lý giỏ hàng **Ưu tiên:** Must.
Mô tả: Khách thêm món đã chọn, điều chỉnh số lượng, xóa món và xem số lượng/tổng tiền của giỏ.
AC-FR03.1: Hai món cùng sản phẩm, size và cấu hình/số phần topping được gộp thành một dòng; nếu một trong các thuộc tính này khác nhau, chúng nằm ở các dòng riêng.

AC-FR03.2: Tăng, giảm hoặc xóa một dòng cập nhật số lượng dòng, tổng số món và tổng tiền ngay trên giao diện.

AC-FR03.3: Giỏ được giữ khi tải lại trang trong cùng trình duyệt; dữ liệu giỏ không được coi là đơn đã tạo cho đến khi máy chủ xác nhận tạo đơn.

AC-FR03.4: Khi khách đăng nhập, giỏ khách được hợp nhất với giỏ tài khoản theo quy tắc gộp ở AC-FR03.1; không làm mất hoặc nhân đôi món ngoài số lượng đã chọn.

AC-FR03.5: Số lượng phải là số nguyên dương trong giới hạn nghiệp vụ; giá client tính chỉ để hiển thị và sẽ được máy chủ xác minh khi đặt đơn.

FR-04 – Kiểm tra giỏ và khả năng đáp ứng nguyên liệu **Ưu tiên:** Must.
Mô tả: Trước khi đặt hàng, hệ thống xác minh sản phẩm, size, topping, công thức, giá và nguyên liệu cho toàn bộ giỏ.
AC-FR04.1: API từ chối sản phẩm không tồn tại/ngừng bán, size không hợp lệ, topping không khả dụng hoặc dữ liệu số lượng sai; phản hồi chỉ ra lỗi có thể xử lý.

AC-FR04.2: Hệ thống cộng định lượng công thức của mọi dòng trong giỏ theo từng nguyên liệu rồi so sánh với tồn khả dụng; nếu thiếu, trả nguyên liệu/nhu cầu thiếu đủ để giao diện giải thích lý do không thể tiếp tục.

AC-FR04.3: Khi giỏ hợp lệ, API trả kết quả kiểm tra và giá tham khảo được tính từ dữ liệu máy chủ.

AC-FR04.4: Gọi kiểm tra giỏ không tạo đơn, không giữ nguyên liệu và không trừ tồn.

FR-05 – Đăng ký và xác minh tài khoản **Ưu tiên:** Must.
Mô tả: Người dùng đăng ký bằng thông tin hợp lệ và hoàn tất xác minh email nếu chính sách xác minh đang bật.
AC-FR05.1: Đăng ký hợp lệ tạo tài khoản với email duy nhất; email đã được dùng bị từ chối bằng thông báo phù hợp.

AC-FR05.2: Mật khẩu được băm trước khi lưu và không xuất hiện trong phản hồi API hoặc log ứng dụng.

AC-FR05.3: Khi xác minh email được bật, mã xác minh có thời hạn và giới hạn thử; mã sai/hết hạn không xác minh tài khoản, mã đúng chỉ xác minh tài khoản dự kiến.

AC-FR05.4: Người dùng có thể yêu cầu gửi lại mã theo giới hạn chống lạm dụng; phản hồi không tiết lộ thông tin nhạy cảm ngoài chính sách hệ thống.

FR-06 – Đăng nhập, đăng xuất và quản lý hồ sơ **Ưu tiên:** Must.
Mô tả: Khách đăng nhập, đăng xuất và xem/cập nhật thông tin hồ sơ của mình.
AC-FR06.1: Thông tin đăng nhập hợp lệ tạo phiên xác thực; sai email/mật khẩu hoặc tài khoản không đáp ứng điều kiện xác thực bị từ chối.

AC-FR06.2: Phiên xác thực được gửi bằng cookie bảo vệ phù hợp; endpoint cần đăng nhập từ chối yêu cầu thiếu phiên, sai phiên hoặc phiên hết hạn.

AC-FR06.3: API xem hồ sơ chỉ trả thông tin tài khoản hiện tại; cập nhật hồ sơ chỉ tác động đến tài khoản đó và kiểm tra định dạng dữ liệu đầu vào.

AC-FR06.4: Đăng xuất xóa cookie phiên trên trình duyệt; yêu cầu tiếp theo không còn được xác thực bằng cookie đã xóa.

FR-07 – Khôi phục và đổi mật khẩu **Ưu tiên:** Should.
Mô tả: Người dùng có thể yêu cầu khôi phục mật khẩu và thay mật khẩu trong phiên đã xác thực.
AC-FR07.1: Yêu cầu quên mật khẩu không cho phép người không có quyền truy cập tài khoản thay mật khẩu nếu thiếu token/mã hợp lệ.

AC-FR07.2: Mã hoặc liên kết đặt lại có thời hạn và chỉ được sử dụng theo quy tắc một lần; mã sai/hết hạn bị từ chối.

AC-FR07.3: Mật khẩu mới đáp ứng chính sách mật khẩu, được băm khi lưu; mật khẩu cũ không được trả về.

AC-FR07.4: Đổi mật khẩu yêu cầu xác thực tài khoản và thông tin bắt buộc theo chính sách hệ thống.

FR-08 – Tạo đơn hàng **Ưu tiên:** Must.
Mô tả: Khách đã đăng nhập gửi giỏ hợp lệ để tạo đơn chờ thanh toán.

AC-FR08.1: API từ chối khách chưa đăng nhập, giỏ rỗng, số lượng không hợp lệ, món hoặc tùy chọn sai, hay thiếu nguyên liệu. Yêu cầu bị từ chối không để lại đơn dở dang.

AC-FR08.2: Máy chủ tải lại giá và quy tắc khuyến mãi từ dữ liệu tin cậy, tự tính tổng tiền; chỉnh sửa giá/tổng tiền phía trình duyệt không làm thay đổi số tiền được ghi nhận.

AC-FR08.3: Khi hợp lệ, đơn và các dòng món được lưu cùng việc giữ nguyên liệu trong một transaction; phản hồi có orderId, tổng tiền, trạng thái PENDING và thời điểm hết hạn giữ nguyên liệu.

AC-FR08.4: Lỗi ở bất kỳ bước ghi nào làm rollback các thay đổi của transaction; không tạo đơn một phần hoặc giữ nguyên liệu một phần.

AC-FR08.5: Chi tiết món, giá và snapshot công thức cần thiết được lưu để đơn đã tạo không bị thay đổi hồi tố khi sản phẩm/công thức được chỉnh sửa sau đó.

FR-09 – Thanh toán đơn hàng **Ưu tiên:** Must.
Mô tả: Khách thanh toán đơn thuộc sở hữu của mình bằng thẻ mô phỏng hoặc VietQR qua PayOS khi đã cấu hình.
AC-FR09.1: Checkout hiển thị mã đơn, số tiền lấy từ máy chủ, phương thức được hỗ trợ và thời gian còn lại nếu đơn có thời hạn.

AC-FR09.2: API checkout yêu cầu xác thực và kiểm tra quyền sở hữu. Yêu cầu POST thanh toán phải có Idempotency-Key. API kiểm tra trạng thái và hạn trước khi xử lý lượt thanh toán mới.

AC-FR09.3: Kết quả thành công chỉ ghi nhận một thanh toán thành công, chuyển đơn sang PAID, tiêu thụ nguyên liệu đã giữ và phát sinh quyền lợi điểm/voucher theo quy tắc.

AC-FR09.4: Kết quả thất bại được lưu/hiển thị là thất bại, không trừ tồn thực và chỉ cho thử lại khi đơn còn hạn, trạng thái cho phép.

AC-FR09.5: Gửi lại cùng thao tác với cùng idempotency key không tạo thanh toán, không tiêu thụ tồn và không cộng điểm lần thứ hai.

AC-FR09.6: Thanh toán thành công hiển thị biên nhận gồm mã đơn, các dòng món, tổng tiền, phương thức và trạng thái.

AC-FR09.7: VietQR được ghi nhận khi webhook có chữ ký hợp lệ và khớp giao dịch, hoặc khi API đối soát PayOS xác nhận link khớp đơn và đã nhận đủ tiền. Mở mã QR chỉ hiển thị thông tin thanh toán.

FR-10 – Hết hạn đơn chờ thanh toán và giải phóng nguyên liệu **Ưu tiên:** Must.
Mô tả: Hệ thống xử lý đơn hết hạn, xác minh giao dịch VietQR nếu có link và giải phóng nguyên liệu khi đủ điều kiện hủy.
AC-FR10.1: Thời hạn giữ nguyên liệu cho đơn mới là 2 phút từ thời điểm tạo đơn, theo thời gian do máy chủ ghi nhận.

AC-FR10.2: Sau khi hết hạn, API chặn lượt thanh toán mới. Đơn chưa nhận tiền được hủy và trả kho đúng một lần. Nếu đã cấp link VietQR, chỉ hủy khi PayOS xác nhận link đã đóng và chưa nhận tiền.

AC-FR10.3: Gọi lặp endpoint kiểm tra trạng thái hoặc tác vụ dọn đơn hết hạn không giải phóng nguyên liệu nhiều lần và không làm tồn khả dụng tăng sai.

AC-FR10.4: Trước khi hết hạn, lỗi thanh toán không tự giải phóng nguyên liệu; khách có thể thử lại nếu đơn vẫn ở trạng thái cho phép.

AC-FR10.5: Khi hết hạn mà PayOS xác nhận đã nhận đủ và đơn còn giữ kho, hệ thống hoàn tất đơn. Nếu PayOS báo nhận thiếu, dữ liệu không khớp hoặc chưa thể đối soát, hệ thống giữ nguyên trạng thái và lượng giữ để kiểm tra tiếp.

FR-11 – Lịch sử, chi tiết và trạng thái đơn của khách **Ưu tiên:** Must.
Mô tả: Khách đã đăng nhập xem lịch sử, chi tiết và trạng thái cập nhật của đơn thuộc tài khoản mình.
AC-FR11.1: Danh sách lịch sử chỉ bao gồm đơn của user đang đăng nhập; tài khoản chưa có đơn nhận danh sách rỗng và thông báo phù hợp trên giao diện.

AC-FR11.2: Khách xem được chi tiết gồm mã đơn, thời gian, món/size/topping, số lượng, tổng tiền và trạng thái.

AC-FR11.3: Khách không thể đọc đơn của người khác bằng cách thay orderId; API từ chối truy cập mà không tiết lộ nội dung đơn.

AC-FR11.4: Giao diện phản ánh trạng thái do máy chủ trả về và làm mới dữ liệu khi đơn còn đang xử lý.

FR-12 – Quản lý trạng thái đơn của cửa hàng **Ưu tiên:** Must.
Mô tả: ADMIN xem các đơn cần xử lý và chuyển trạng thái theo luồng hợp lệ.
AC-FR12.1: Endpoint quản trị yêu cầu phiên xác thực và vai trò ADMIN; user thường bị từ chối.

AC-FR12.2: ADMIN xem danh sách đơn có lọc trạng thái/ngày/từ khóa và phân trang; kết quả đi kèm tổng số phù hợp với điều kiện lọc.

AC-FR12.3: ADMIN xem số lượng đơn theo trạng thái và số liệu doanh thu theo phạm vi thời gian đã chọn; giá trị tính từ các trạng thái được tính là đã thanh toán.

AC-FR12.4: Chuyển PAID → PREPARING → READY → COMPLETED được chấp nhận; chuyển trạng thái không hợp lệ hoặc cập nhật đơn không tồn tại bị từ chối và không làm thay đổi trạng thái.

AC-FR12.5: Đơn PENDING chưa thanh toán không được đưa vào danh sách pha chế như đơn cần thực hiện.

FR-13 – Khuyến mãi và voucher **Ưu tiên:** Should.
Mô tả: Khách xem và áp dụng khuyến mãi/voucher hợp lệ; máy chủ kiểm tra điều kiện trước khi giảm giá và ghi đơn.
AC-FR13.1: Khuyến mãi có thể được tìm/xem theo trạng thái công khai; dữ liệu trả về không bao gồm bí mật quản trị.

AC-FR13.2: Mã không tồn tại, chưa bắt đầu, hết hạn, không hoạt động hoặc không đạt giá trị đơn tối thiểu bị từ chối với lý do phù hợp.

AC-FR13.3: Máy chủ tính mức giảm và giới hạn giảm theo cấu hình; client không thể tự sửa số tiền giảm.

AC-FR13.4: Điều kiện voucher được xác minh lại khi tạo đơn/thanh toán theo quy tắc nghiệp vụ, tránh áp dụng mã đã hết hiệu lực sau khi xem giỏ.

AC-FR13.5: Voucher giới hạn lượt sử dụng không bị ghi nhận vượt giới hạn trong các yêu cầu đồng thời.

FR-14 – Điểm thưởng và đổi thưởng **Ưu tiên:** Could.
Mô tả: Khách hàng đã đăng nhập xem điểm và đổi điểm lấy phần thưởng nếu đủ điều kiện.
AC-FR14.1: API điểm thưởng yêu cầu xác thực và chỉ trả số dư/lịch sử của user hiện tại.

AC-FR14.2: Đổi thưởng yêu cầu phần thưởng còn hiệu lực và số dư đủ; nếu không đủ điều kiện, điểm không bị trừ.

AC-FR14.3: Cộng, đổi hoặc hoàn điểm tạo giao dịch sổ điểm tương ứng; cùng một đơn/thao tác không phát sinh giao dịch trùng khi được gửi lại.

AC-FR14.4: Điểm tích lũy từ mua hàng chỉ được cấp khi điều kiện thanh toán thành công được đáp ứng; hủy/hoàn theo nghiệp vụ không làm số dư sai lệch.

FR-15 – Quản lý nguyên liệu và công thức **Ưu tiên:** Should.
Mô tả: ADMIN xem và sửa nguyên liệu, nhập kho, quản lý định lượng cho món và topping.
AC-FR15.1: Chỉ ADMIN có quyền xem và sửa nguyên liệu, công thức qua API quản trị.

AC-FR15.2: Nguyên liệu có mã duy nhất, tên, đơn vị, trạng thái theo dõi tồn và các trường số lượng hợp lệ; mã trùng hoặc số lượng âm bị từ chối.

AC-FR15.3: Nhập kho ghi tăng số lượng thực có và không ghi đè lượng đang giữ; thao tác không làm tồn âm hoặc mất dữ liệu do cập nhật đồng thời.

AC-FR15.4: Công thức sản phẩm gắn với một product và size; công thức topping gắn với topping; mỗi định lượng lớn hơn 0 và tham chiếu nguyên liệu hợp lệ.

AC-FR15.5: Lưu/sửa công thức được phản ánh trong lần kiểm tra giỏ tiếp theo; đơn đã tạo giữ snapshot cần thiết để lịch sử không đổi.

AC-FR15.6: Nguyên liệu không theo dõi tồn không bị coi là hết hàng chỉ vì không có số lượng tồn.

FR-16 – Quản lý sản phẩm và khuyến mãi ở vai trò ADMIN **Ưu tiên:** Could.
Mô tả: ADMIN quản lý trạng thái khả dụng sản phẩm và thông tin khuyến mãi theo quyền được phép.
AC-FR16.1: Thay đổi trạng thái khả dụng sản phẩm cập nhật khả năng xuất hiện/được đặt trên luồng khách hàng sau khi dữ liệu được làm mới.

AC-FR16.2: ADMIN có thể xem, tạo hoặc bật/tắt khuyến mãi theo trường dữ liệu bắt buộc; dữ liệu sai bị từ chối.

AC-FR16.3: Thay đổi admin được ghi vào cơ sở dữ liệu và có kết quả thành công/lỗi rõ ràng; lỗi không trả phản hồi thành công giả.

1.2.2 Yêu cầu phi chức năng

Các ngưỡng hiệu năng và vận hành là mục tiêu nghiệm thu đề xuất; nhóm cần ghi cấu hình môi trường và dữ liệu kiểm thử khi đo.

NFR-01 – Hiệu năng **Ưu tiên:** Should.
Yêu cầu: Các thao tác phổ biến phải phản hồi trong thời gian chấp nhận được ở tải mục tiêu.
AC-NFR01.1: Với 100 phiên người dùng đồng thời trên môi trường kiểm thử đã ghi cấu hình, ít nhất 95% yêu cầu đọc menu/chi tiết/lịch sử hoàn tất trong 2 giây.

AC-NFR01.2: Trong cùng điều kiện, ít nhất 95% yêu cầu ghi giỏ/đơn/thanh toán hoàn tất trong 3 giây, không tính thời gian người dùng chờ tương tác ngoài hệ thống.

AC-NFR01.3: Báo cáo kiểm thử ghi cấu hình máy, kích thước dữ liệu, số phiên đồng thời, số mẫu, p50/p95/p99 và lỗi; không nghiệm thu chỉ dựa trên một lần đo thủ công.

NFR-02 – Bảo mật và quyền riêng tư **Ưu tiên:** Must.
Yêu cầu: Xác thực, phân quyền và bảo vệ dữ liệu phải được áp dụng nhất quán ở giao diện và API; kiểm tra quyền phải được thực hiện ở phía máy chủ.
AC-NFR02.1: Mật khẩu được lưu bằng thuật toán băm mật khẩu phù hợp; mật khẩu, token bí mật và thông tin nhạy cảm không xuất hiện trong phản hồi API hoặc log.

AC-NFR02.2: Cookie phiên có HttpOnly; production bật Secure và thiết lập SameSite phù hợp với cách triển khai.

AC-NFR02.3: API yêu cầu đăng nhập từ chối request không có phiên hợp lệ; endpoint admin từ chối user không có vai trò ADMIN.

AC-NFR02.4: Kiểm thử phân quyền chứng minh user A không đọc/sửa đơn, hồ sơ hoặc dữ liệu riêng của user B bằng cách thay ID trong request.

AC-NFR02.5: Mọi đầu vào được kiểm tra kiểu, định dạng, miền giá trị và quyền sở hữu; giá/giảm giá/trạng thái không được tin từ client.

AC-NFR02.6: Môi trường production chỉ truyền dữ liệu xác thực qua HTTPS; secret triển khai được cung cấp qua cấu hình bí mật, không commit vào repository.

NFR-03 – Tính toàn vẹn và nhất quán dữ liệu **Ưu tiên:** Must.
Yêu cầu: Giao dịch tạo đơn, giữ tồn, thanh toán, tiêu thụ tồn, voucher và điểm không được để lại trạng thái một phần.
AC-NFR03.1: Kiểm thử nhiều đơn đồng thời với nhu cầu nguyên liệu vượt tồn không làm tồn khả dụng âm và không chấp nhận lượng bán vượt nguyên liệu có thể dùng.

AC-NFR03.2: Khi lỗi xảy ra giữa một giao dịch tạo đơn/thanh toán, các thay đổi liên quan được rollback hoặc phục hồi theo cùng một trạng thái nhất quán.

AC-NFR03.3: Gửi lại cùng khóa không tạo thêm thanh toán, trừ thêm tồn hoặc ghi thêm điểm.

AC-NFR03.4: Mọi chuyển trạng thái đơn không hợp lệ bị từ chối ở API, kể cả khi gửi request trực tiếp không qua giao diện.

NFR-04 – Khả dụng và phục hồi dịch vụ **Ưu tiên:** Should.
Yêu cầu: Lỗi phụ thuộc hoặc sự cố triển khai cần được phát hiện; quy trình khôi phục không được âm thầm báo thành công.
AC-NFR04.1: Mục tiêu production đề xuất là khả dụng ít nhất 99% mỗi tháng, không tính thời gian bảo trì đã thông báo; tính theo log/monitoring có thời điểm và mẫu số rõ ràng.

AC-NFR04.2: Health check phải phát hiện API không sẵn sàng và, nếu được cấu hình, không thể kết nối cơ sở dữ liệu.

AC-NFR04.3: Một lần phát hành chỉ được đánh dấu thành công sau 5 lần kiểm tra health liên tiếp đạt yêu cầu; nếu thất bại phải báo lỗi và cung cấp bước xử lý/rollback.

AC-NFR04.4: Đơn, payment và tồn kho đã commit phải còn tồn tại sau khi restart tiến trình ứng dụng.

NFR-05 – Khả năng mở rộng **Ưu tiên:** Should.
Yêu cầu: Web/API có thể tăng tải mà không phụ thuộc dữ liệu nghiệp vụ trong bộ nhớ của một tiến trình.
AC-NFR05.1: Có thể chạy nhiều bản sao web/API cùng dùng cơ sở dữ liệu chung mà không nhân đôi đơn, thanh toán, giữ tồn hoặc điểm trong các kiểm thử lặp/đồng thời.

AC-NFR05.2: Dữ liệu giỏ trên trình duyệt không được dùng làm nguồn dữ liệu duy nhất cho đơn, thanh toán hoặc tồn kho.

AC-NFR05.3: Kết quả kiểm thử tăng tải được so sánh với các ngưỡng NFR-01; mọi giới hạn tài nguyên hoặc suy giảm hiệu năng phải được nêu trong báo cáo.

NFR-06 – Khả dụng giao diện và khả năng tiếp cận **Ưu tiên:** Should.
Yêu cầu: Luồng chính phải dễ nhận biết và sử dụng trên thiết bị di động, bàn phím và các trạng thái lỗi.
AC-NFR06.1: Các màn hình menu, chi tiết, giỏ, đăng nhập, checkout và lịch sử không tràn ngang ở viewport rộng 360 CSS px.

AC-NFR06.2: Các thao tác chính có thể thực hiện bằng bàn phím; focus nhìn thấy được; trường nhập có nhãn; ảnh chức năng có văn bản thay thế phù hợp.

AC-NFR06.3: Trạng thái loading, empty, error, success và lỗi nghiệp vụ được thông báo bằng nội dung, không chỉ khác màu sắc.

AC-NFR06.4: Lỗi nhập liệu chỉ rõ trường hoặc hành động cần sửa; thông báo không làm mất dữ liệu người dùng đã nhập nếu không cần thiết.

NFR-07 – Bảo trì và kiểm thử **Ưu tiên:** Should.
Yêu cầu: Thay đổi phải có thể kiểm tra lặp lại và pipeline cần chặn thay đổi làm hỏng chất lượng đã thống nhất.
AC-NFR07.1: Mỗi pull request chạy lint, build và unit test cho ứng dụng liên quan; API chạy Prisma generate trước bước cần Prisma Client.

AC-NFR07.2: Nghiệp vụ quan trọng gồm phân quyền, tính giá, chuyển trạng thái, idempotency và xử lý tồn có test tự động cho trường hợp thành công và thất bại.

AC-NFR07.3: Test lỗi phải trả exit code thất bại để CI chặn merge; không được chuyển lỗi thành cảnh báo hoặc kết quả thành công.

AC-NFR07.4: Hướng dẫn cài đặt, cấu hình môi trường và chạy test phải khớp cấu trúc repository hiện hành.

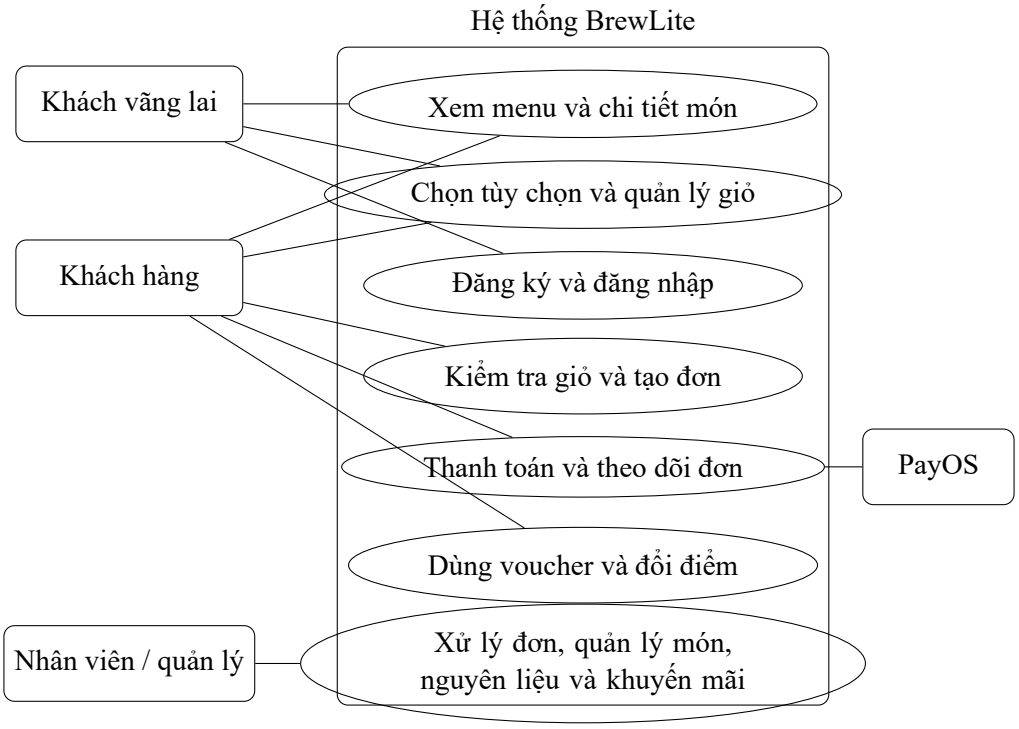
NFR-08 – Sao lưu và khôi phục dữ liệu **Ưu tiên:** Could.
Yêu cầu: Dữ liệu nghiệp vụ quan trọng phải có quy trình sao lưu và kiểm tra khả năng khôi phục trước khi triển khai môi trường sử dụng thật.
AC-NFR08.1: Tài liệu vận hành nêu cách tạo backup PostgreSQL, nơi lưu an toàn, thời hạn lưu và người phụ trách; backup không chứa trong Git.

AC-NFR08.2: Trước migration phá hủy dữ liệu, có backup được xác minh và hướng dẫn phục hồi.

AC-NFR08.3: Ít nhất một lần kiểm thử khôi phục trên môi trường không production xác nhận có thể đọc lại dữ liệu đã backup; ghi thời gian và kết quả kiểm thử.

1.2.3 Sơ đồ use-case tổng quát

Hình dưới biểu diễn các nhóm chức năng và actor ở mức tổng quát. Đặc tả luồng chính, thay thế và ngoại lệ của use-case trọng tâm thuộc Task 1.3.



Ràng buộc actor: Xem menu và chuẩn bị giỏ không cần đăng nhập. Tạo đơn, thanh toán và xem lịch sử cần xác thực. Quản trị yêu cầu vai trò `ADMIN`. Hệ thống kết nối dịch vụ email và PayOS khi có cấu hình tương ứng.

1.2.4 Các giả định cần nhóm xác nhận

Ngưỡng tải, thời gian phản hồi và mục tiêu khả dụng trong các NFR là mục tiêu đề xuất; quy tắc voucher/điểm và chính sách hủy/hoàn cần được thống nhất trước khi nghiệm thu. Secret, webhook key và thông tin kết nối production không nằm trong tài liệu hoặc repository.

1.3 Task 1.3 – Đặc tả use-case trọng tâm

1.3.1 UC-01 – Đặt và thanh toán đơn hàng

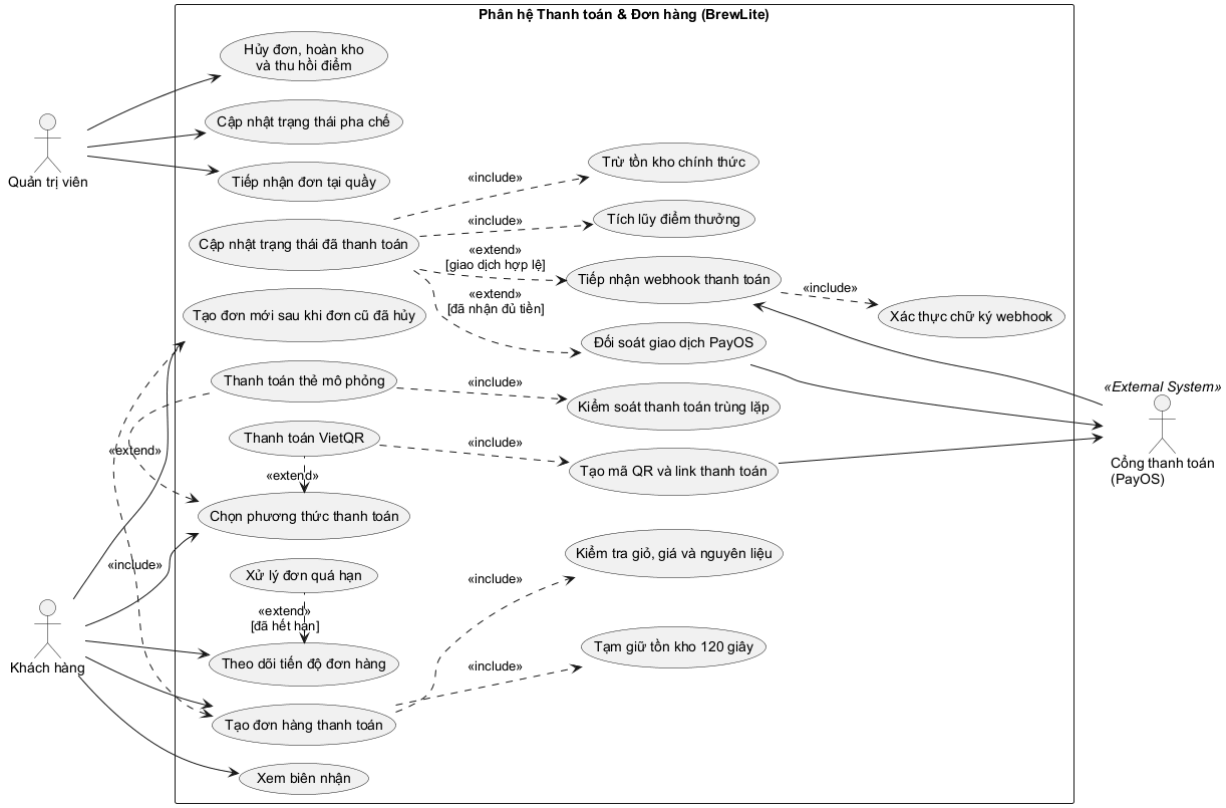
Mục tiêu: Khách chuyển giỏ đồ uống thành đơn có mã, thanh toán trong thời hạn và nhận biên nhận. Cửa hàng chỉ chuẩn bị đơn đã thanh toán; nguyên liệu được kiểm tra theo công thức để tránh bán vượt lượng có thể dùng.

Actor chính: Khách hàng.
Actor hỗ trợ: PayOS tạo link VietQR và gửi webhook xác nhận giao dịch. Nhánh thẻ dùng cổng mô phỏng trong ứng dụng.
Trigger: Khách bấm “Tiến hành thanh toán” ở giỏ hàng.
Tiền điều kiện: Giỏ có ít nhất một món. Khách cần phiên đăng nhập hợp lệ trước khi gọi API tạo đơn. Sản phẩm, size, topping và công thức có dữ liệu trên máy chủ; đủ nguyên liệu và voucher

hợp lệ được kiểm tra trong luồng.
Hậu điều kiện thành công: Đơn có `orderId`, trạng thái `PAID`, một payment thành công; lượng nguyên liệu đã giữ được tiêu thụ đúng một lần, điểm/voucher được ghi theo quy tắc, khách xem được biên nhận và giỏ cục bộ được xóa.
Hậu điều kiện không thành công: Giỏ sai hoặc thiếu nguyên liệu không tạo đơn. Đơn chờ hoặc thanh toán thẻ lỗi giữ nguyên liệu đến khi thanh toán thành công hoặc đủ điều kiện hủy. Đơn VietQR có link chỉ trả kho khi PayOS xác nhận link đã đóng và chưa nhận tiền. Đơn chưa trả tiền không được cộng điểm hoặc đưa vào pha chế.

1.3.2 Sơ đồ use-case của UC-01

Sơ đồ thể hiện vai trò của khách hàng, PayOS và quản trị viên trong phân hệ Thanh toán và Đơn hàng. UC-01 tập trung vào các thao tác đặt hàng, thanh toán và xác nhận giao dịch.



Sơ đồ được lưu riêng ở `docs/diagrams/uc01-order-checkout.puml` và xuất thành ảnh trước khi đưa vào báo cáo. **Ranh giới:** UC-01 gồm kiểm tra giỏ, tạo đơn, giữ/trả nguyên liệu, thanh toán và hiển thị kết quả. Thao tác tiếp nhận, cập nhật trạng thái pha chế và hủy đơn đã thanh toán thuộc luồng quản trị riêng. Khi hủy đơn đã thanh toán ở trạng thái cho phép, BrewLite hoàn kho và thu hồi điểm đã cấp trong giới hạn số dư hiện tại. Việc mở QR hoặc bấm "Đã chuyển tiền" không tự xác nhận đã thanh toán.

1.3.3 Luồng chính – thanh toán thành công

1. Khách xem giỏ, điều chỉnh món/số lượng rồi bấm "Tiến hành thanh toán".

2. Nếu đã đăng nhập, giao diện gọi `POST /api/cart/validate`. API đọc lại sản phẩm, size, topping, giá, công thức và nguyên liệu cho toàn bộ giỏ.

3. Khi giỏ hợp lệ, giao diện gọi `POST /api/orders`. Máy chủ tính lại tiền/voucher; trong một transaction kiểm tra lại tồn, giữ nguyên liệu và lưu đơn cùng các dòng món.

4. API trả `orderId`, `PENDING`, tổng tiền và thời hạn hai phút. Giao diện mở `/checkout?orderId=...`; đồng hồ dùng hạn do máy chủ trả về, không tính lại từ lúc mở trang.

5. Với thẻ mô phỏng, ứng dụng gửi `POST /api/checkout` với `orderId`, `method=CARD` và `Idempotency-Key`. Với VietQR, khách quét mã PayOS. API xác minh chữ ký và dữ liệu giao dịch khi nhận webhook, hoặc đối soát trực tiếp link và số tiền qua PayOS khi đọc trạng thái.

6. Khi thanh toán được xác nhận, máy chủ ghi payment, chuyển đơn thành `PAID`, tiêu thụ nguyên liệu đã giữ và xử lý điểm/voucher. Giao diện đọc trạng thái, hiện biên nhận và xóa giỏ cục bộ.

1.3.4 Luồng thay thế

A1 – Chưa đăng nhập (bước 2)
Giao diện yêu cầu đăng nhập; API tạo đơn từ chối request không có phiên. Sau khi đăng nhập, khách tiếp tục với giỏ của mình; chưa có đơn hay nguyên liệu bị giữ.

A2 – Giỏ/voucher không hợp lệ (bước 2–3)
Giỏ rỗng, sản phẩm ngừng bán, size/topping/số lượng sai, công thức thiếu, voucher hết hiệu lực hoặc thiếu nguyên liệu được báo lỗi. Khách sửa và thử lại; `POST /api/orders` kiểm tra lại giá/tồn trước khi commit.

A3 – Chọn phương thức (bước 5)
Khách đổi giữa thẻ mô phỏng và VietQR khi đơn còn hạn. Việc chọn phương thức không tự tạo đơn mới hay ghi nhận thanh toán. Nếu PayOS đã nhận tiền hoặc chưa xác nhận đóng link an toàn, API không chạy lượt thẻ mới.

A4 – Thẻ mô phỏng thất bại (bước 5)
Payment ghi `FAILED`, đơn chuyển `PAYMENT_FAILED`; nguyên liệu vẫn giữ đến hạn và khách được thử lại bằng khóa giao dịch mới.

A5 – Tạo lại sau hết hạn (bước 4–5)
Sau khi máy chủ xác nhận đơn cũ đã hủy, giao diện cho một lượt tạo lại: đơn cũ `CANCELLED` đã trả nguyên liệu; gọi lại `POST /api/orders`, kiểm tra tồn hiện tại và nhận `orderId` mới cùng hạn hai phút. Giới hạn một lượt hiện do giao diện thực hiện, chưa phải ràng buộc ở API.

1.3.5 Luồng ngoại lệ

E1 – Hết hạn
Sau hai phút, API chặn lượt thanh toán mới. Khi đọc trạng thái hoặc chạy tác vụ nền, API hủy đơn chưa thanh toán và trả lượng giữ. Nếu có link VietQR, API đối soát trước khi hủy: nhận đủ thì hoàn tất đơn còn giữ kho; link đã đóng và chưa nhận tiền thì hủy; chưa xác minh được thì giữ nguyên đơn.

E2 – Gửi lặp

Cùng `Idempotency-Key` trả payment đã có, không trừ nguyên liệu/cộng điểm hai lần; dùng cùng khóa cho đơn khác bị từ chối.

E3 – Sai quyền hoặc trạng thái

Thiếu phiên, orderId của người khác, đơn đã `PAID`/`CANCELLED` hoặc quá hạn bị từ chối trước khi ghi nhận payment mới.

E4 – Lỗi giao dịch/tranh chấp tồn

Lỗi giữ nguyên liệu, ghi đơn hoặc ghi thanh toán làm rollback phần thay đổi của transaction; giao diện báo lỗi và giữ dữ liệu giỏ để thử lại.

E5 – Chưa có tín hiệu PayOS

Trang đọc trạng thái định kỳ hoặc khi khách bấm kiểm tra; API có thể đối soát trực tiếp với PayOS. Nếu đã nhận một phần tiền, dữ liệu không khớp hoặc PayOS chưa phản hồi, trang tiếp tục chờ hoặc báo cần đối soát. Nếu đơn đã hủy và trả kho nhưng PayOS báo có tiền, xử lý qua đối soát riêng.

1.3.6 Liên kết yêu cầu và tiêu chí chấp nhận

Mã	Điều kiện và kết quả cần kiểm tra	Yêu cầu nguồn
`UC01-AC01`	Giỏ hợp lệ tạo một đơn `PENDING`, trả mã/tổng tiền/hạn hai phút; `reserved` tăng đúng công thức.	FR-03, FR-04, FR-08
`UC01-AC02`	Chưa đăng nhập, giỏ sai, voucher không hợp lệ hoặc thiếu nguyên liệu: báo lỗi, không lưu đơn/giữ nguyên liệu.	FR-04, FR-06, FR-08, FR-13
`UC01-AC03`	Sửa giá/tổng tiền ở client không đổi số tiền máy chủ tính và ghi trên đơn.	FR-08, NFR-02
`UC01-AC04`	Thẻ mô phỏng thành công trong hạn: một payment, đơn `PAID`, giảm `onHand`/`reserved`, hiện biên nhận.	FR-09, NFR-03
`UC01-AC05`	VietQR được xác nhận qua webhook hợp lệ hoặc đối soát PayOS xác nhận đã nhận đủ tiền. Mở QR không tự ghi nhận thanh toán.	FR-09, NFR-03
`UC01-AC06`	Gửi lại cùng khóa hoặc request đồng thời không trừ tồn/cộng điểm hai lần.	FR-09, FR-14, NFR-03
`UC01-AC07`	Thẻ lỗi còn hạn được thử lại. Hết hạn thì chặn lượt mới; chỉ hủy và trả kho một lần khi đủ điều kiện theo FR-10.	FR-09, FR-10
`UC01-AC08`	Lượt tạo lại sau hết hạn dùng `orderId` mới, chỉ thành công nếu tồn hiện tại đủ.	FR-04, FR-08, FR-10
`UC01-AC09`	Khách chỉ xem/thanh toán đơn của mình; khi thành công xem được biên nhận và lịch sử đúng trạng thái.	FR-09, FR-11, NFR-02

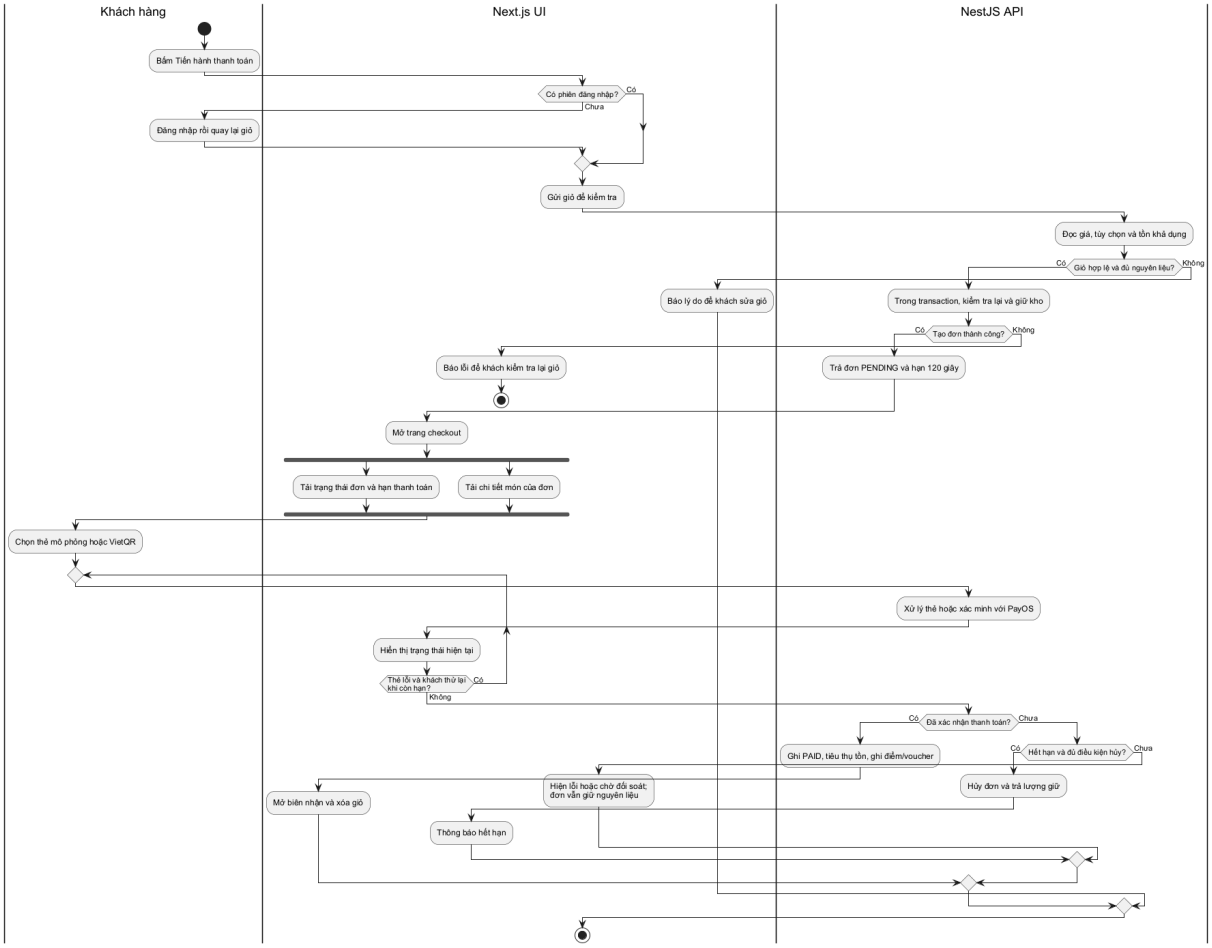
Giới hạn cần nghiệm thu riêng: Quy tắc "chỉ một lần tạo lại" hiện được lưu ở trình duyệt; muốn chặn lách qua thiết bị khác hoặc API cần quan hệ giữa các lượt đơn và kiểm tra tại máy chủ. Nếu đơn đã hủy và trả kho, một giao dịch VietQR đến muộn cần đối soát riêng. Nếu đơn vẫn giữ kho, API có thể hoàn tất khi xác minh PayOS đã nhận đủ tiền.

2 Task 2 – Mô hình hóa hệ thống

2.1 Task 2.1 – Activity diagram

2.1.1 Luồng đặt và thanh toán

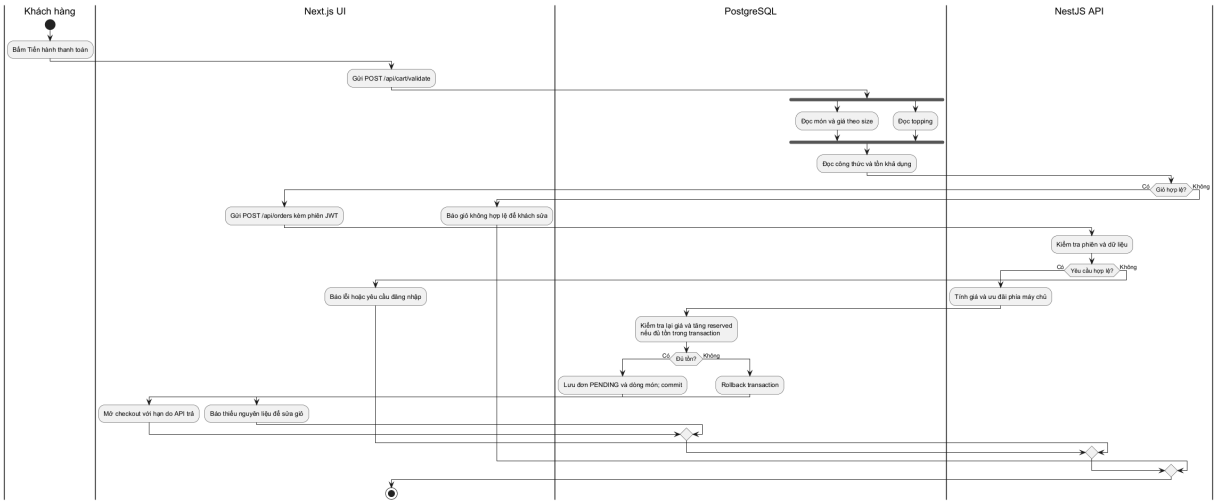
Sơ đồ theo UC-01 từ giỏ hàng đến kết quả thanh toán. Swimlane phân biệt thao tác của khách, Next.js và NestJS. Khi mở checkout, giao diện tải trạng thái thanh toán và chi tiết đơn song song. Khi giỏ không hợp lệ, khách quay lại sửa giỏ. Đơn thẻ lỗi còn hạn có thể thử lại; đơn hết hạn chỉ trả kho khi đủ điều kiện hủy.



Hình 2.1a. Hoạt động đặt và thanh toán đơn hàng

2.1.2 Kiểm tra giỏ và giữ nguyên liệu

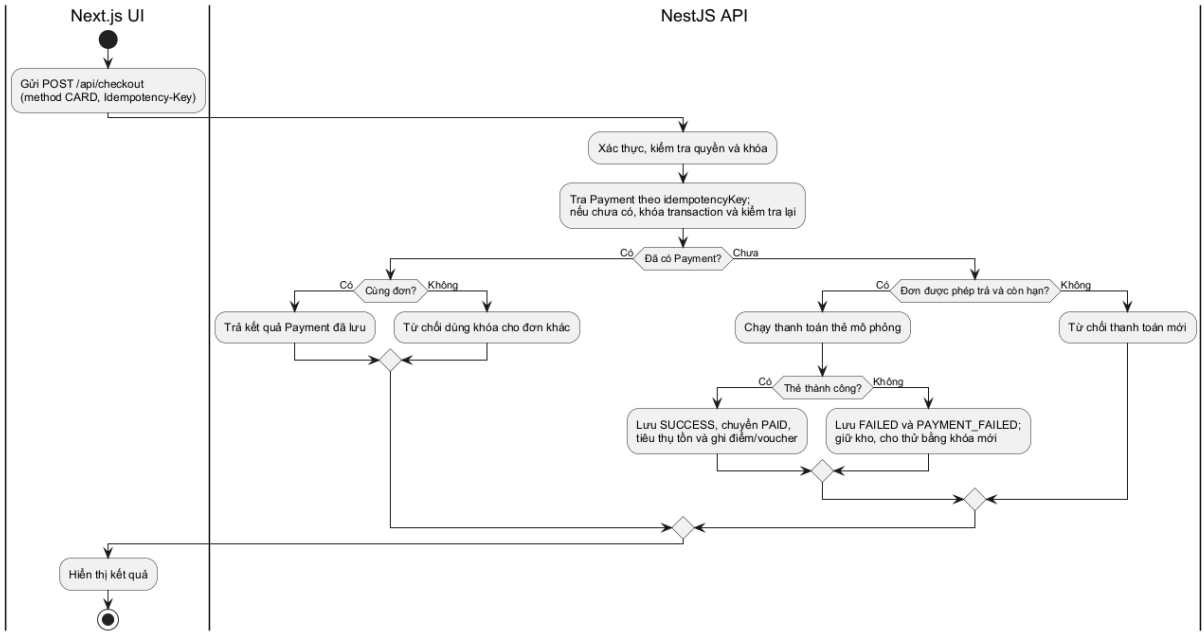
API kiểm tra giỏ ở chế độ chỉ đọc. Hai truy vấn món kèm giá và topping được khởi chạy cùng lúc. API chờ đủ hai kết quả trước khi đọc công thức. Khi tạo đơn, API tính giá và ưu đãi, sau đó kiểm tra lại giá và tăng lượng giữ bằng lệnh cập nhật có điều kiện đủ tồn trong transaction. Đơn, dòng món và lượng giữ được ghi cùng nhau. Khi thiếu nguyên liệu hoặc có lỗi ghi, transaction được hoàn tác.



Hình 2.1b. Kiểm tra giỏ và giữ nguyên liệu

2.1.3 Thanh toán thẻ mô phỏng và chống xử lý lặp

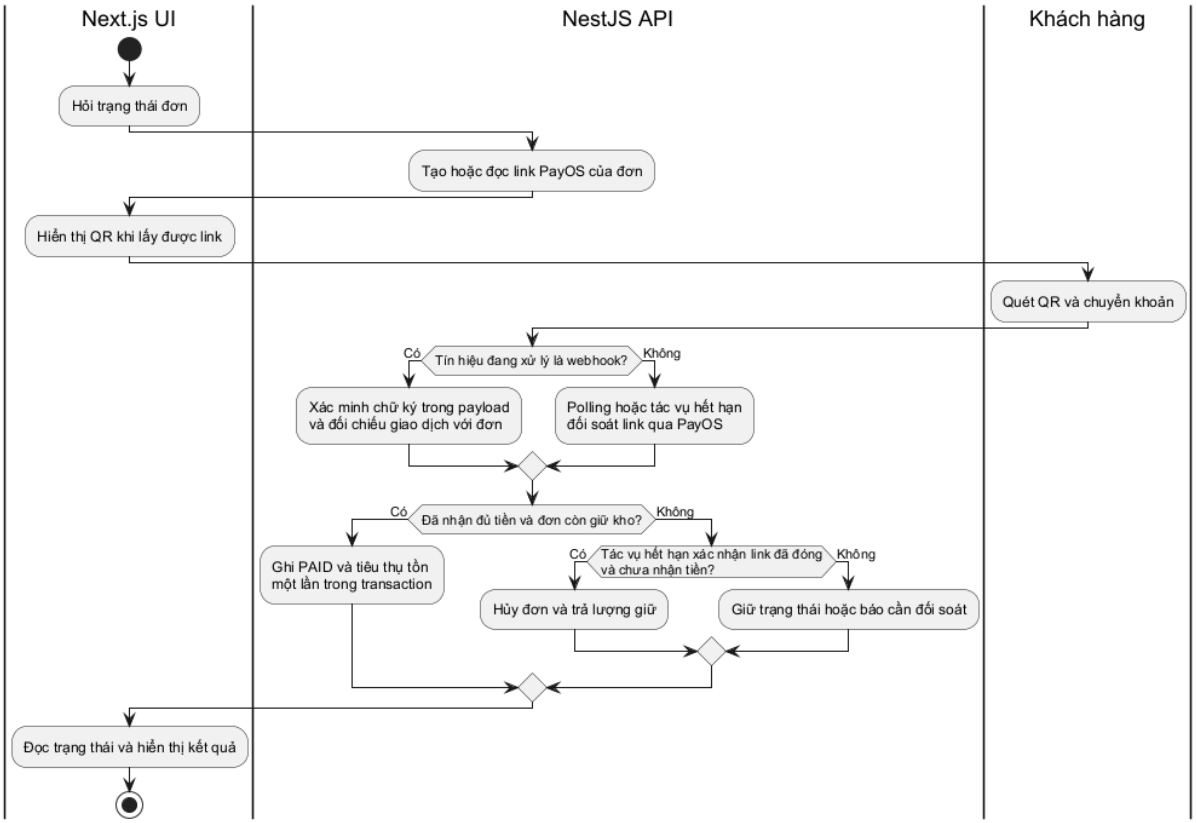
Khóa `Idempotency-Key` gắn với bản ghi `Payment`. Yêu cầu lặp trả kết quả đã ghi. Nếu thẻ thất bại, API ghi `FAILED`, đơn chuyển sang `PAYMENT_FAILED` và giữ kho đến khi đơn được thanh toán hoặc đủ điều kiện hủy.



Hình 2.1c. Xử lý thẻ mô phỏng

2.1.4 VietQR, webhook và đối soát

Webhook có chữ ký hợp lệ hoặc kết quả đối soát trực tiếp có thể xác nhận đã nhận đủ tiền. Khi hết hạn, đơn có link PayOS chỉ được hủy và trả kho nếu link đã đóng và chưa nhận tiền. Nếu PayOS chưa xác nhận hoặc số tiền chưa khớp, API giữ nguyên đơn để đối soát.

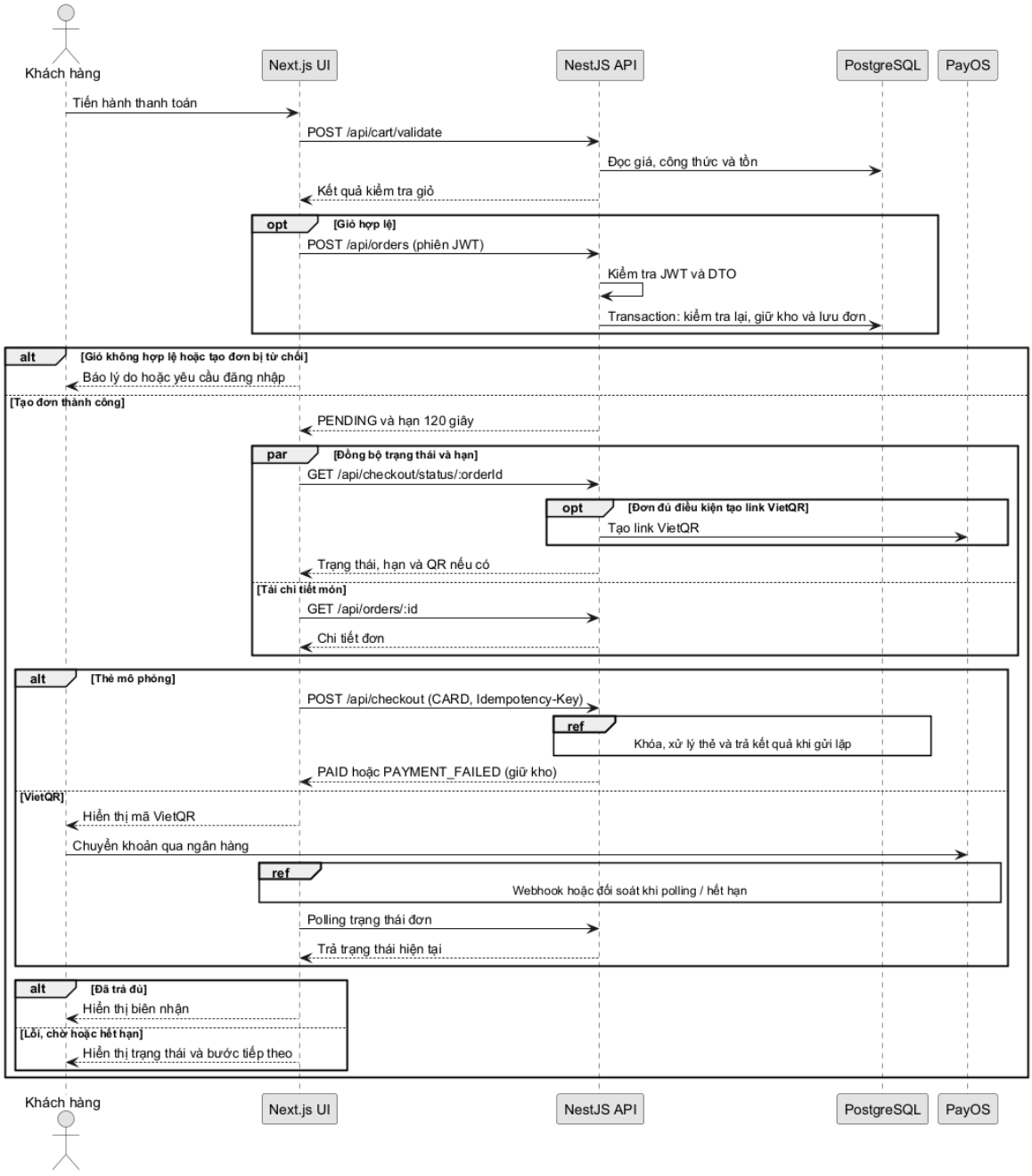


Hình 2.1d. Xác minh giao dịch VietQR và xử lý hết hạn

2.2 Task 2.2 – Sequence diagram

2.2.1 Luồng UC-01 từ giỏ đến biên nhận

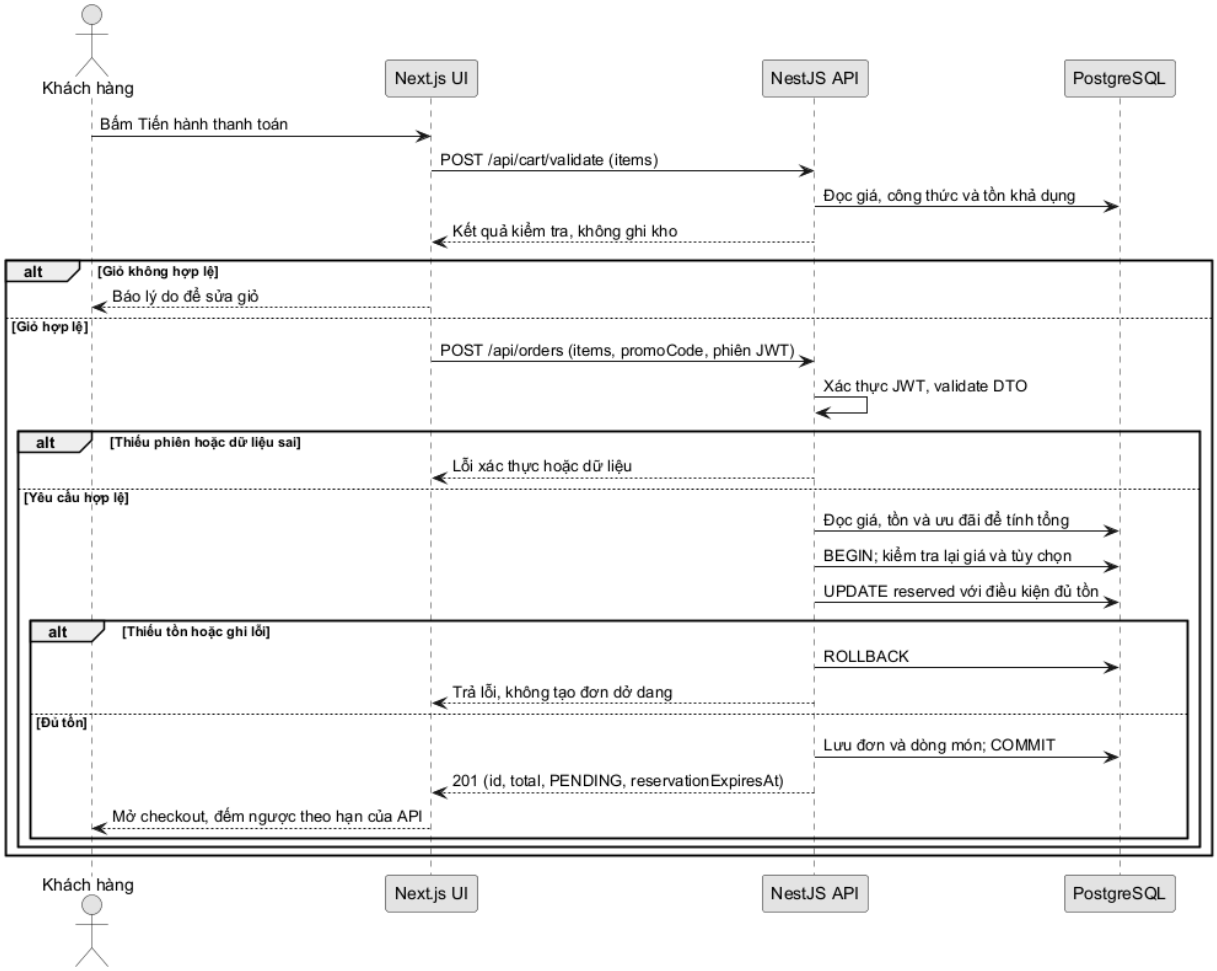
Năm lifeline gồm khách hàng, Next.js UI, NestJS API, PostgreSQL và PayOS. Sơ đồ thể hiện kiểm tra giỏ, tạo đơn với phiên JWT, transaction giữ nguyên liệu, hai nhánh thanh toán và kết quả hiển thị. Khi mở checkout, giao diện tải trạng thái và chi tiết đơn song song như trong Activity Diagram. Các khung tham chiếu dẫn đến sơ đồ chi tiết.



Hình 2.2a. Trình tự đặt và thanh toán đơn hàng

2.2.2 Tạo đơn và tạm giữ nguyên liệu

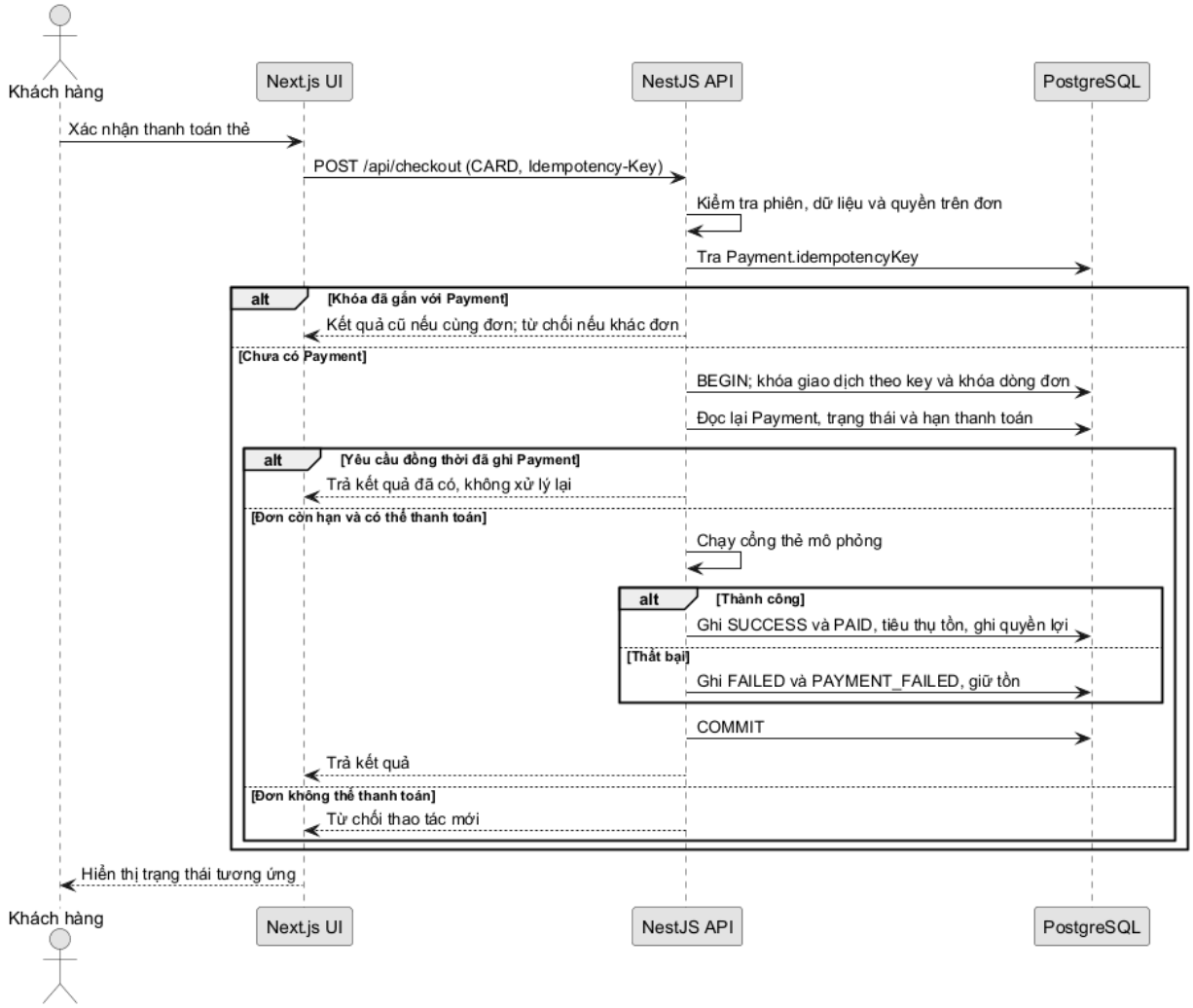
Bước kiểm tra giỏ chỉ đọc dữ liệu. API tạo đơn yêu cầu JWT, kiểm tra DTO, tính giá và ưu đãi ở máy chủ, kiểm tra lại giá rồi giữ nguyên liệu bằng cập nhật có điều kiện trong transaction. API trả `id`, `total`, trạng thái `PENDING` và `reservationExpiresAt`.



Hình 2.2b. Kiểm tra dữ liệu, khóa nguyên liệu và tạo đơn

2.2.3 Chống xử lý lặp khi thanh toán thẻ

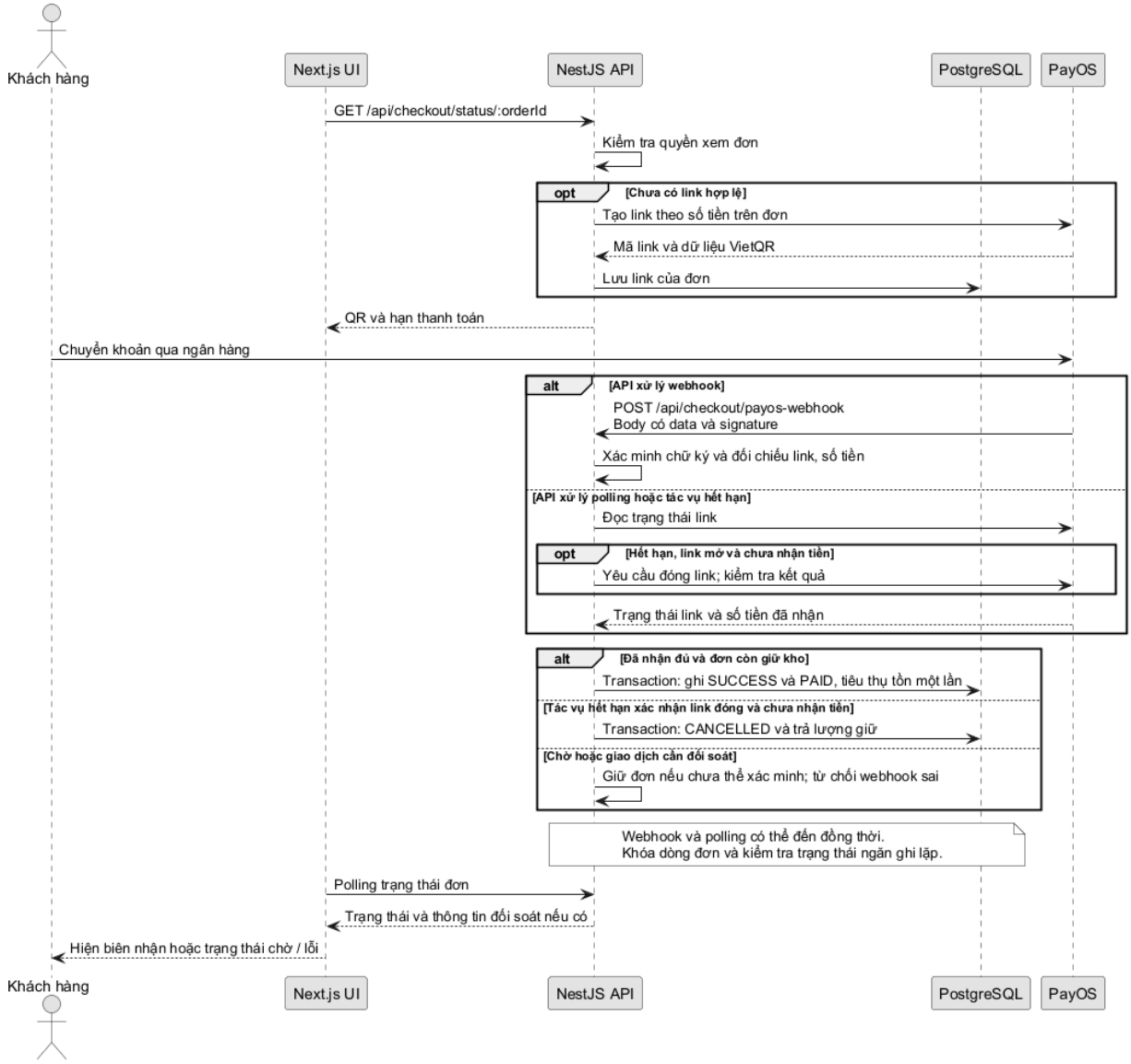
API tra `Payment.idempotencyKey`. Nếu chưa có kết quả, transaction dùng khóa theo key và khóa dòng đơn, rồi kiểm tra lại để xử lý các yêu cầu đồng thời. Kết quả trả về dựa trên bản ghi `Payment`. Lượt thử sau thất bại dùng khóa mới nếu đơn còn hạn.



Hình 2.2c. Khóa transaction và trả lại kết quả đã ghi

2.2.4 Thanh toán VietQR và đối soát PayOS

PayOS gửi webhook có `data` và `signature` trong body. API dùng SDK PayOS kiểm tra chữ ký và đối chiếu giao dịch với đơn. Polling hoặc tác vụ hết hạn cũng có thể đọc trực tiếp trạng thái link. Webhook hợp lệ và polling đồng thời chỉ hoàn tất đơn một lần.



Hình 2.2d. Xác thực webhook, polling và đối soát khi hết hạn

2.3 Task 2.3 – Class diagram

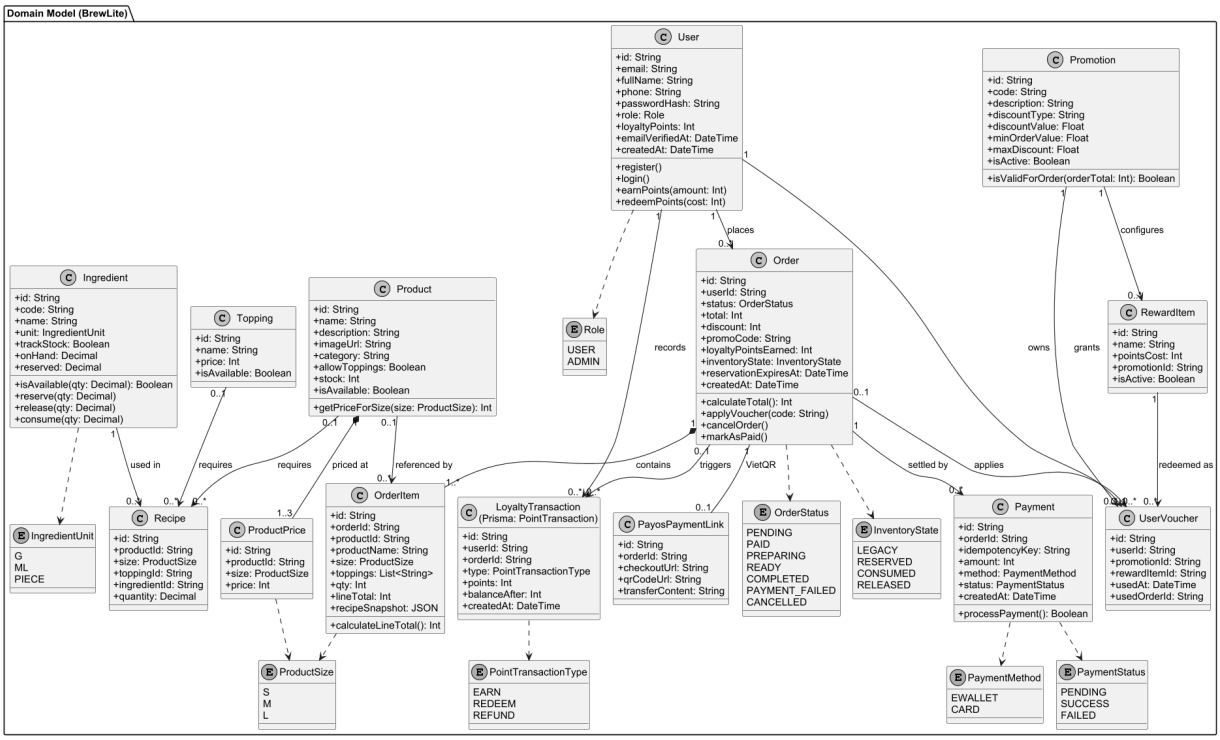
2.3.1 Mục tiêu và phạm vi mô hình hóa mức Domain

Class Diagram mô hình hóa cấu trúc tĩnh ở mức miền nghiệp vụ (Domain Model) của hệ thống BrewLite, phản ánh trung thực lược đồ cơ sở dữ liệu PostgreSQL (thông qua Prisma ORM) và các quy tắc nghiệp vụ trong SRS.

Bên cạnh 7 thực thể bắt buộc theo rubric của đề bài (**User**, **Product**, **Order**, **OrderItem**, **Payment**, **Promotion**, **LoyaltyTransaction**), nhóm bổ sung các lớp miền phản ánh nét đặc thù pha chế và quản lý nguyên liệu của BrewLite:

- **Ingredient (Nguyên liệu):** Quản lý tồn kho thực tế ($onHand$) và lượng tạm giữ ($reserved$) theo từng đơn vị ($g, ml, piece$).
- **Recipe (Công thức định mức):** Cầu nối giữa món nước / topping và nguyên liệu thô để phục vụ tính toán tồn kho tự động.
- **ProductPrice (Giá theo size):** Quản lý đơn giá theo từng kích cỡ ly (S, M, L).
- **Topping:** Danh mục các món thêm vào đồ uống (trân châu, thạch, đào miếng).
- **RewardItem:** Danh mục quà, số điểm cần để đổi và khuyến mãi tương ứng.
- **UserVoucher:** Quản lý trạng thái mã ưu đãi mà khách hàng sở hữu hoặc đã sử dụng.
- **PayosPaymentLink:** Lưu link và mã VietQR theo đơn để checkout xác minh giao dịch hoặc đối soát khi webhook đến trễ.

2.3.2 Sơ đồ lớp miền (Domain Class Diagram)



Hình: Class diagram ở mức domain của hệ thống BrewLite (File nguồn: `docs/diagrams/class-domain.puml`)

2.3.3 Mô tả các thực thể chính và thuộc tính

1. **User:** Quản lý danh tính khách hàng và quản trị viên (`Role: USER, ADMIN`), số điểm thưởng tích lũy (`loyaltyPoints`) và trạng thái xác minh email.
2. **Order:** Trung tâm của quy trình đặt hàng, chứa `status` (`OrderStatus`), `total`, `discount`, trạng thái tồn kho (`inventoryState: LEGACY, RESERVED, CONSUMED, RELEASED`) và mốc thời gian hết hạn 120 giây (`reservationExpiresAt`).
3. **OrderItem:** Thành phần cấu thành đơn hàng (quan hệ Composition $1 \longrightarrow 1..*$). Chứa số lượng, size ly, mảng topping đã chọn, tổng tiền dòng và `recipeSnapshot` lưu vết công thức tại thời điểm đặt.
4. **Payment:** Ghi nhận thanh toán với `idempotencyKey` duy nhất chống lặp, phương thức thanh toán (`PaymentMethod: CARD, EWALLET`) và trạng thái giao dịch (`PaymentStatus: PENDING, SUCCESS, FAILED`).
5. **Ingredient & Recipe:** Hỗ trợ bài toán kiểm soát tồn kho thời gian thực. Mỗi `Recipe` định mức số lượng nguyên liệu cần dùng cho từng `Product` (theo size) hoặc `Topping`.
6. **Promotion & LoyaltyTransaction:** Quản lý chiến dịch giảm giá và lịch sử biến động điểm thưởng (`EARN, REDEEM, REFUND`) có thể gắn với một đơn hàng. Trong Prisma, lớp sổ điểm được lưu dưới tên `PointTransaction`.

2.3.4 Quan hệ và bội số (Multiplicity)

- **User – Order ($1 \longrightarrow 0..*$):** Một người dùng có thể đặt nhiều đơn hàng; mỗi đơn hàng thuộc về đúng một người dùng.
- **Order – OrderItem ($1 \longrightarrow 1..*$):** Quan hệ hợp thành (Composition). Một đơn hàng phải có ít nhất 1 dòng món; nếu xóa đơn hàng thì các dòng món con bị xóa theo.
- **Product – ProductPrice ($1 \longrightarrow 1..3$):** Một sản phẩm có từ 1 đến 3 mức giá tương ứng với các kích cỡ S, M, L.
- **Order – PayosPaymentLink ($1 \longrightarrow 0..1$):** Mỗi đơn có tối đa một link VietQR lưu trong hệ thống.
- **Order – Payment ($1 \longrightarrow 0..*$):** Một đơn hàng có thể có nhiều lượt thử thanh toán (ví dụ lượt 1 lỗi thì ghi `FAILED`, lượt 2 thành công ghi `SUCCESS`).
- **Order – UserVoucher ($0..1 \longrightarrow 0..1$):** Mỗi đơn hàng chỉ được áp dụng tối đa một mã giảm giá hợp lệ.
- **Ingredient – Recipe ($1 \longrightarrow 0..*$):** Một loại nguyên liệu thô có thể được sử dụng trong nhiều công thức pha chế khác nhau.

3 Task 3 – Thiết kế kiến trúc

3.1 Task 3.1 – Phương pháp và quyết định kiến trúc

Nhóm mô tả BrewLite theo C4 ở ba mức gồm ngữ cảnh, container và thành phần API. Web Next.js phục vụ giao diện mobile-first trên Vercel. API NestJS chạy trên Render theo kiến trúc modular monolith và truy cập PostgreSQL trên Supabase qua Prisma. CI và quy trình phát hành dùng migration được lưu trong Git.

Khi tạo đơn, API giữ nguyên liệu theo công thức. Thanh toán CARD dùng cổng mô phỏng. Luồng VietQR dùng PayOS và đối soát trước khi giải phóng kho của đơn hết hạn. PostgreSQL lưu thay đổi đơn, kho và điểm trong một transaction khi xác nhận thanh toán. API kiểm tra vai trò và quyền sở hữu đơn, ghi log bằng NestJS và kiểm tra kết nối cơ sở dữ liệu qua `/api/health`.

Thư mục `docs/architecture/` trong repo lưu sơ đồ, luồng xử lý, cấu hình theo môi trường và các đánh đổi trong tệp `system-architecture.md`. Thư mục con `adr/` lưu sáu quyết định kiến trúc.

3.2 Task 3.2 – Deployment diagram

Khung nội dung: nhóm sẽ bổ sung tạo phẩm và minh chứng của task này.

4 Task 4 – Triển khai Sprint 1

4.1 Task 4.1 – Thiết lập repository

Mã nguồn Assignment 2 được quản lý trong repository BrewLite với nhánh tích hợp `main`. `apps/web` chứa ứng dụng Next.js, `apps/api` chứa NestJS và Prisma, còn `docs` lưu hồ sơ dự án. Ở thư mục gốc, `README.md` giới thiệu cấu trúc và cách kiểm thử, `.env.example` ghi cấu hình mẫu và `.gitignore` loại trừ các tệp môi trường cục bộ. Web và API có cấu hình mẫu riêng theo ứng dụng.

Nhánh, pull request và commit Thay đổi được thực hiện trên nhánh theo công việc và đưa vào `main` qua pull request. Ví dụ, nhánh `payments` được tích hợp qua PR #1. Thông điệp commit nên nêu một thay đổi cụ thể, như `Fix API lockfile for npm ci`. Pull request ghi mục đích, phần bị ảnh hưởng và cách kiểm thử để người review có thể tái hiện.

Product Backlog theo sprint

- **Sprint 1** thiết lập repo và hồ sơ kỹ thuật, xây API danh mục và chi tiết món, hoàn thiện menu mobile-first, chuẩn bị dữ liệu demo và CI cho web/API.
- **Sprint 2** hoàn thiện tùy chọn món, giỏ, xác thực, đặt đơn, thanh toán và theo dõi đơn. Tích hợp kiểm thử, phát hành production và chuẩn bị bàn giao.

Danh sách trên ghi lại phạm vi của hai sprint. Báo cáo chưa có liên kết đến issue board của nhóm.

Definition of Done Một hạng mục hoàn tất khi tiêu chí chấp nhận được kiểm tra, thay đổi mã và tài liệu liên quan được review qua pull request, các kiểm tra CI đều đạt và có thể tìm lại minh chứng từ PR hoặc tài liệu truy vết. Với thay đổi production, cần ghi nhận kết quả smoke test và cách quay lại bản trước. Cấu hình bắt buộc các kiểm tra trước khi merge được đối chiếu ở Task 4.4.

4.2 Task 4.2 – Quản lý tài liệu

Hồ sơ yêu cầu, mô hình hóa và kiến trúc được quản lý cùng mã nguồn trong `docs/`. Báo cáo LaTeX dùng `docs/requirements/task1/main.tex` để ghép các mục từ tệp con. Các tạo phẩm

chi tiết được tổ chức như sau.

- **Task 1** lưu bối cảnh và phạm vi, SRS và đặc tả use-case trong `docs/requirements/`.
- **Task 2** lưu mã nguồn PlantUML và các bản xuất PNG/SVG của sơ đồ trong `docs/diagrams/`. Báo cáo tham chiếu đến bản xuất tương ứng.
- **Task 3** lưu tài liệu kiến trúc và sáu ADR trong `docs/architecture/`. `docs/deployment.md` mô tả môi trường triển khai, migration và rollback.

Khi thay đổi yêu cầu, sơ đồ hoặc kiến trúc, pull request ghi lý do và cập nhật các tạo phẩm bị ảnh hưởng. Với sơ đồ, cập nhật cả nguồn và ảnh kết xuất trong cùng thay đổi. `docs/traceability-matrix.md` liên kết mã yêu cầu với use-case hoặc user story, thiết kế, API và ca kiểm thử.

4.3 Task 4.3 – MVP Sprint 1

MVP Sprint 1 cho phép khách mở web trên điện thoại, xem menu và mở chi tiết món từ dữ liệu do API cung cấp. `GET /api/products` trả danh sách món và hỗ trợ lọc theo danh mục. `GET /api/products/:id` trả chi tiết theo mã món và trả HTTP 404 nếu mã không tồn tại. Hai endpoint nằm trong `products.controller.ts` của module sản phẩm.

Trang chủ Next.js hiển thị danh sách món qua `AllDrinksScroll`. Màn hình có trạng thái tải, thông báo khi danh sách rỗng và thông báo lỗi khi tải thất bại. Khi có dữ liệu, khách có thể chọn món để mở trang `/product/[id]`. Prisma seed và các script tạo danh mục, nạp tồn demo chuẩn bị dữ liệu trên PostgreSQL local theo `docker/README.md`.

Ở bộ kiểm thử hiện tại, `AllDrinksScroll.spec.ts` kiểm tra các trạng thái của menu. `app.e2e-spec.ts` gọi API menu và chi tiết món trước khi đi tiếp đến luồng đặt đơn. Các lệnh chạy và dữ liệu kiểm thử được ghi tại `tests/README.md`.

4.4 Task 4.4 – Continuous Integration

Workflow `.github/workflows/ci.yml` tự chạy khi mở hoặc cập nhật pull request vào `main`. Hai job `Build & Check Backend` và `Build & Check Frontend` dùng Node.js 22, cài phụ thuộc bằng `npm ci`, rồi lần lượt chạy lint, unit test và build. Backend tạo Prisma Client trước khi lint. Job `API integration and order E2E` dựng PostgreSQL 16 tạm, áp migration và chạy test tích hợp cùng E2E qua API.

Minh chứng trên PR thanh toán

- PR #1 tích hợp nhánh `payments` vào `main`.
- CI lượt #6 phát hiện lỗi ở bước `npm ci` của hai job dùng API, tại commit `7438dd1`.
- CI lượt #7 đạt cả ba job sau khi cập nhật `apps/api/package-lock.json` ở commit `f5dcc2d`.

GitHub hiện báo `main` chưa được bảo vệ. Để đáp ứng tiêu chí chặn merge khi CI thất bại, cần bật branch protection hoặc ruleset trên `main` và đặt cả ba job trên làm required checks. Quality gate của workflow phát hành đã chặn bước triển khai nếu CI thất bại.

5 Task 5 – Triển khai Sprint 2 và bàn giao

5.1 Task 5.1 – Hoàn thiện MVP

Sprint 2 nối luồng xem menu với giỏ hàng, đặt đơn, thanh toán và nhận đồ tại quầy. Web triển khai các màn hình chi tiết món, giỏ, xác thực, checkout và lịch sử đơn. NestJS quản lý nghiệp vụ, trạng thái đơn và dữ liệu trong PostgreSQL.

1. **Chọn món và cập nhật giỏ.** Khách chọn size, topping phù hợp và số lượng. Giỏ cho sửa số lượng, xóa món và xem tổng tiền. Web hiển thị tiền tạm tính và giảm giá. API kiểm tra lại giá, tùy chọn và tồn khả dụng trước khi chấp nhận đơn.
2. **Xác thực và tạo đơn.** Khách đăng ký, xác minh email bằng OTP và đăng nhập. API dùng JWT để bảo vệ thao tác đặt hàng và kiểm tra quyền truy cập đơn. Đơn mới có trạng thái `PENDING`. API giữ nguyên liệu theo công thức trong thời hạn 120 giây.
3. **Thanh toán.** Kênh CARD dùng cổng mô phỏng. Lựa chọn ví/QR dùng VietQR qua PayOS khi có cấu hình. Bộ test có thể giả lập kết quả PayOS bằng link và webhook ký bằng khóa thử. API dùng khóa giao dịch và kiểm tra trạng thái đơn để tránh ghi nhận trùng. Thanh toán thành công chuyển đơn sang `PAID` và tiêu thụ nguyên liệu đã giữ.
4. **Xác nhận và theo dõi.** Khách xem biên nhận, trạng thái hiện tại và lịch sử đơn. Nhân viên xử lý các bước pha chế, sẵn sàng và hoàn tất trên giao diện quản trị.
5. **Xử lý lỗi.** Web hiển thị lỗi nhập liệu, thiếu nguyên liệu hoặc lỗi thanh toán. API kiểm tra DTO, quyền truy cập, trạng thái đơn và dữ liệu giao dịch. Khi đơn QR hết hạn, API đối soát với PayOS trước khi quyết định hủy và trả phần nguyên liệu đã giữ.

Chi tiết luồng thành công và ngoại lệ nằm ở UC-01. Bộ kiểm thử liên quan được liệt kê ở Task 5.2 và `tests/README.md`.

5.2 Task 5.2 – Tích hợp và kiểm thử

GitHub Actions chạy lint, unit test và build cho web và API. Job tích hợp dùng PostgreSQL 16 riêng, chạy Prisma migration trước khi kiểm thử.

- **Unit test** kiểm tra xác thực, tính giá, khuyến mãi, điểm thưởng, chuyển trạng thái đơn và giỏ hàng. Các ca phía web còn kiểm tra trạng thái tải, danh sách rỗng và lỗi của menu.
- **Integration test** gọi API tạo đơn và thanh toán trên DB thử nghiệm. Các ca bao gồm tranh chấp nguyên liệu, quyền sở hữu đơn, thử lại bằng khóa giao dịch, hết hạn trả kho và đối soát PayOS.
- **E2E qua API** đi từ menu và chi tiết món đến đăng nhập, kiểm tra giỏ, tạo đơn, thanh toán CARD mô phỏng và xác nhận đơn `PAID` trong lịch sử. Ca VietQR riêng dùng link giả lập và webhook có chữ ký thử.
- **Tái tạo local** dùng `docker-compose.yml` cho web, API và PostgreSQL. Prisma migration tạo schema. Seed và hai lệnh tạo danh mục, nạp tồn demo chuẩn bị dữ liệu trên DB local trống.
- **Health check** tại `/api/health` kiểm tra kết nối PostgreSQL. Sau khi triển khai, smoke test cũng gọi endpoint này qua proxy của web.

Danh sách ca, lệnh chạy và các bước nghiệm thu trình duyệt nằm trong `tests/README.md`.

5.3 Task 5.3 – Continuous Delivery/Deployment

Workflow `.github/workflows/cd.yml` tự chạy khi có push vào `main`, bao gồm commit sau khi merge PR. Workflow cũng hỗ trợ tag `v*` và chạy thủ công. Các bước phát hành chạy theo thứ tự sau.

1. **Quality gate.** Gọi lại CI trên cùng commit để chạy lint, unit test, build, kiểm thử tích hợp và E2E. Chỉ chuyển sang đóng gói khi toàn bộ job kiểm tra đạt.
2. **Đóng gói.** Build hai Docker image web và API, gắn tag bằng commit SHA và đẩy lên GitHub Container Registry.
3. **Kiểm tra đích và migration.** Dùng secrets từ GitHub Environment `production`. Kiểm tra đích Render/Vercel và xác nhận baseline migration cùng bản sao lưu. Sau đó chạy `prisma migrate deploy` trên Supabase.
4. **Deploy và smoke test.** Render triển khai API từ SHA phát hành. Vercel CLI triển khai web từ cùng checkout. Workflow kiểm tra `/api/health`, `/api/products`, trang web HTTPS và health endpoint qua proxy web. Image GHCR lưu bản đóng gói theo SHA để truy vết.

Nếu rollout lỗi sau khi đã cập nhật một ứng dụng, workflow thử khôi phục bản Vercel trước đó hoặc commit Render trước đó, tùy bước đã thành công. Rollback schema và dữ liệu PostgreSQL được xử lý riêng theo `docs/deployment.md`. Lượt phát hành của commit `55e5bbe` ghi nhận CI, publish image, migration, deploy và smoke test đều đạt.

5.4 Task 5.4 – Triển khai và bàn giao

- **Mã nguồn trên GitHub** `https://github.com/PhucTruong11/brewlite-assignment2`.
- **Web production** `https://brewlite-frontend.vercel.app/`.
- **API production** `https://brewlite-api-omni.onrender.com/api`.
- **Health check trực tiếp** `https://brewlite-api-omni.onrender.com/api/health`.
- **Health check qua web** `https://brewlite-frontend.vercel.app/api/health`.

Kiểm tra ngày 13/11/2026 ghi nhận HTTP 200 ở trang web, API health, API danh sách món và health check qua proxy web.

Web Next.js chạy trên Vercel và gọi API NestJS trên Render qua proxy cùng origin. Render kết nối PostgreSQL Supabase qua TLS bằng thông tin xác thực. Trình duyệt truy cập dữ liệu qua API. Cấu hình production và bí mật nằm trong các môi trường của Render, Vercel và GitHub Actions. Repository cung cấp tệp mẫu để chỉ ra tên biến cần cấu hình. Log CD dùng để đối chiếu kết quả migration, deploy và smoke test.

Hướng dẫn tái tạo và demo Người nghiệm thu có thể clone repository, dùng `README.md` để xem lệnh test và build, dùng `docker/README.md` để khởi động web, API và PostgreSQL local, và dùng `docs/deployment.md` để xem quy trình release và rollback. Kịch bản demo đi từ menu đến chọn size/topping, kiểm tra giỏ, đăng nhập, tạo đơn, thanh toán CARD mô phỏng, mở biên nhận và kiểm tra lịch sử đơn. Có thể trình bày thêm luồng VietQR khi kênh PayOS đã cấu hình.

Minh chứng bàn giao Phiên bản production đối chiếu được theo commit `55e5bbe` và log phát hành. Tag phiên bản và release notes chưa thấy trên GitHub. Link video hoặc biên bản demo trực tiếp, Sprint Review và Sprint Retrospective cần đính kèm khi nộp nếu nhóm đã có các tài liệu này. Cấu hình giới hạn mạng của Supabase cần được xác nhận từ dashboard dự án.